Hafta 2 — Detay ve Derin Referans

> ## ⚠️ Bu dosya sınav kapsamı DEĞİLDİR.
>
> Buradaki hiçbir konu vize veya finalde sorulmaz. Bu dosya, **merak eden ve projesinde daha ileri gitmek isteyen** öğrenci içindir. Sınava `H02-ders-notu.md` dosyasından çalışın.

İçindekiler

1. MVC deseninin kısa tarihi
2. `IActionResult` ile `ActionResult<T>` farkı
3. Routing motoru içeride nasıl çalışır
4. Middleware'i sınıf olarak yazmak
5. `Use`, `Run`, `Map` farkları
6. DI kabının içi ve karşıt desenler
7. Area (alan) kavramı
8. Kaynakça

---

1. MVC Deseninin Kısa Tarihi

MVC, web için icat edilmedi. **1979**'da Trygve Reenskaug tarafından Smalltalk-80 için tasarlandı — masaüstü grafik arayüzlerinde veriyi, görünümü ve kullanıcı etkileşimini ayırmak içindi.

Dönem	Ortam	MVC'nin hâli
1979	Smalltalk-80	Özgün hâli: View, Model'i **doğrudan gözlemler**
1990'lar	Masaüstü (MFC, Swing)	Controller, olay dinleyicisine dönüştü
2004	Ruby on Rails	Web'e uyarlandı; "convention over configuration" yaygınlaştı
2009	ASP.NET MVC 1.0	Microsoft ekosistemine geldi
2016	ASP.NET Core MVC	Çapraz platform, DI çekirdeğe gömülü

Web'de ne değişti: Özgün MVC'de View, Model'i sürekli gözler ve değişince kendini günceller. Web'de bu mümkün değil — **HTTP stateless'tir** (Hafta 1). Her istek yeni bir View üretir. Dolayısıyla web MVC'si, özgün desenin **tek yönlü** bir uyarlamasıdır:

```
İstek → Controller → Model → Controller → View → Yanıt
```

View, Model'i gözlemez; Controller ona verir. Bu yüzden bazıları web MVC'sine **"Model-View-Adapter"** demeyi tercih eder.

---

2. `IActionResult` ile `ActionResult<T>` Farkı

Ders notunda `IActionResult` kullandık. Web API'de (Hafta 12) bir alternatif göreceksiniz:

```
// 1) IActionResult — dönüş tipi belirsiz
public IActionResult Getir(int id)
{
    var kitap = Bul(id);
    if (kitap is null) return NotFound();
    return Ok(kitap);
}

// 2) ActionResult<T> — dönüş tipi belirtilmiş
public ActionResult<Kitap> Getir(int id)
{
    var kitap = Bul(id);
    if (kitap is null) return NotFound();
    return kitap;              // örtük olarak Ok(kitap)
}
```

İkincisinin avantajı: OpenAPI/Swagger belgelendirmesi dönüş tipini **otomatik** bilir, ayrıca derleyici yanlış tip döndürmenizi engeller.

MVC sayfalarında neden kullanmıyoruz? Çünkü bir sayfa HTML döndürür; "hangi tip" sorusunun anlamı yoktur. `ActionResult<T>` API için tasarlanmıştır.

---

3. Routing Motoru İçeride Nasıl Çalışır

`app.UseRouting()` ve `app.MapControllerRoute(...)` aslında iki farklı iş yapar:

Bileşen	İşi
`UseRouting`	Gelen adresi, kayıtlı **endpoint** listesiyle eşleştirir ve sonucu `HttpContext`'e yazar
`MapControllerRoute`	Endpoint'leri **kaydeder** (hangi kalıp hangi controller/action'a)
`UseEndpoints` / `Map…`	Eşleşen endpoint'i **çalıştırır**

.NET 6'dan itibaren `UseEndpoints` çağrısı örtük hâle geldi; `MapControllerRoute` doğrudan `app` üzerinde çağrılabiliyor.

Route değerleri sözlükte tutulur

Eşleşme sonucunda bir `RouteValueDictionary` oluşur:

```
/Kitap/Detay/42  →  { controller = "Kitap", action = "Detay", id = "42" }
```

Dikkat: **değerler** `string`. `int id` parametresine dönüşüm, Model Binding sırasında yapılır (Hafta 4). Bu yüzden `abc` değeri `int`e çevrilemediğinde hata **routing'de değil, binding'de** oluşur — kısıt (`{id:int}`) koymak hatayı routing aşamasına çeker ve 404 yapar.

Endpoint'leri listelemek

Hangi route'ların kayıtlı olduğunu görmek için:

```
app.MapGet("/rotalar", (IEnumerable<EndpointDataSource> kaynaklar) =>
    string.Join("\n", kaynaklar.SelectMany(k => k.Endpoints).Select(e => e.DisplayName)));
```

Hata ayıklarken çok işe yarar — "route'um neden eşleşmiyor" sorusunun cevabı genelde buradadır.

---

4. Middleware'i Sınıf Olarak Yazmak

Ders notunda satır içi (`app.Use(async …)`) biçimi kullandık. Gerçek projelerde middleware genellikle **sınıf** olarak yazılır — test edilebilir ve yeniden kullanılabilir olsun diye.

```
public class IstekGunlukleyici
{
    private readonly RequestDelegate _sonraki;
    private readonly ILogger<IstekGunlukleyici> _gunluk;

    public IstekGunlukleyici(RequestDelegate sonraki, ILogger<IstekGunlukleyici> gunluk)
    {
        _sonraki = sonraki;
        _gunluk = gunluk;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        _gunluk.LogInformation("→ {Metot} {Yol}", context.Request.Method, context.Request.Path);
        await _sonraki(context);
        _gunluk.LogInformation("← {Kod}", context.Response.StatusCode);
    }
}
```

Kayıt:

```
app.UseMiddleware<IstekGunlukleyici>();
```

Sözleşme (convention): Sınıfın kurucusu `RequestDelegate` almalı ve `InvokeAsync(HttpContext)` metodu bulunmalıdır. Arayüz uygulamak **zorunlu değildir** — çatı bu iki üyeyi yansımayla (reflection) bulur.

> 💡 Alternatif olarak `IMiddleware` arayüzü uygulanabilir; o zaman middleware DI'dan çözülür ve **her istekte yeni örnek** alınabilir. Satır içi ve sınıf biçiminde middleware ise uygulama boyunca **tek örnektir** (singleton gibi) — bu yüzden içinde durum tutmayın.

Uzantı metodu ile düzgün API

```
public static class IstekGunlukleyiciUzantilari
{
    public static IApplicationBuilder UseIstekGunlukleyici(this IApplicationBuilder app)
        => app.UseMiddleware<IstekGunlukleyici>();
}

// kullanım:
app.UseIstekGunlukleyici();
```

ASP.NET Core'un tüm `Use…` metotları tam olarak böyle yazılmıştır.

---

5. `Use`, `Run`, `Map` Farkları

Metot	Davranış
`app.Use(...)`	Pipeline'a halka ekler; `next()` çağırarak devam ettirir
`app.Run(...)`	**Terminal** middleware — `next` almaz, her zaman short-circuit yapar
`app.Map("/yol", ...)`	Belirli bir yol önekine **dallanan** ayrı bir pipeline kurar
`app.MapWhen(kosul, ...)`	Koşula göre dallanır

```
app.Map("/saglik", dal =>
{
    dal.Run(async ctx => await ctx.Response.WriteAsync("iyi"));
});
```

`/saglik` isteği bu dala girer, `"iyi"` döner ve **ana pipeline'a hiç uğramaz** — routing, authorization, hiçbiri çalışmaz. Sağlık kontrolü (health check) uçları genelde böyle kurulur.

> ⚠️ `app.Run` pipeline'ın **sonu** demektir. Yanlışlıkla ortada kullanırsanız arkasındaki her şey ölür — üstelik hata almazsınız. Ders notundaki "short-circuit" tuzağının kardeşidir.

---

6. DI Kabının İçi ve Karşıt Desenler

Kap (container) nasıl çalışır?

`builder.Services` bir `IServiceCollection`'dır — aslında sadece bir **liste**: "şu arayüz istendiğinde şu sınıfı ver, şu yaşam döngüsüyle".

`builder.Build()` çağrıldığında bu liste bir `IServiceProvider`'a **derlenir**. Artık değiştirilemez — ders notundaki "`Build()` sonrası servis eklenemez" kuralının sebebi budur.

Scope (kapsam) nedir?

`AddScoped`'ın "scope"u, ASP.NET Core'da **bir HTTP isteğidir.** Çatı her istek için bir `IServiceScope` açar, istek bitince kapatır ve `IDisposable` servisleri atar.

Arka plan işinde (background service) istek yoktur; scope'u **elle** açmanız gerekir:

```
using var scope = _serviceProvider.CreateScope();
var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
```

Bunu yapmadan singleton bir arka plan servisinden `DbContext` almaya çalışmak, ders notundaki `Cannot consume scoped service from singleton` hatasının sebebidir.

Karşıt desen: Service Locator

```
// KÖTÜ — bağımlılık gizli
public class KitapController : Controller
{
    public IActionResult Index()
    {
        var servis = HttpContext.RequestServices.GetService<IKitapServisi>();
        return View(servis!.Hepsi());
    }
}
```

Bu çalışır ama **bağımlılığı gizler**: sınıfın kurucusuna bakan biri neye ihtiyaç duyduğunu göremez, test yazan biri neyi taklit edeceğini bilemez. Buna **Service Locator karşıt deseni** denir. Kurucu enjeksiyonunu tercih edin.

`GetService` ↔ `GetRequiredService`

Metot	Servis kayıtlı değilse
`GetService<T>()`	`null` döner — **sessiz hata**
`GetRequiredService<T>()`	İstisna fırlatır — **açık hata**

İkincisini tercih edin; ilki sizi `NullReferenceException` avına çıkarır.

7. Area (Alan) Kavramı

Proje büyüdüğünde `Controllers/` klasörü onlarca dosya barındırmaya başlar. **Area**, uygulamayı alt uygulamalara bölmenin yoludur:

```
Areas/
├── Yonetim/
│   ├── Controllers/KitapController.cs
│   └── Views/Kitap/Index.cshtml
└── Magaza/
    ├── Controllers/KitapController.cs
    └── Views/Kitap/Index.cshtml
```

İki tane `KitapController` bir arada durabilir çünkü farklı area'lardadır.

Kayıt:

```
app.MapControllerRoute(
    name: "areas",
    pattern: "{area:exists}/{controller=Home}/{action=Index}/{id?}");
```

Controller'a öznitelik eklenir:

```
[Area("Yonetim")]
public class KitapController : Controller { }
```

Adres: `/Yonetim/Kitap/Index`

> 📌 **Bu derste Area kullanmıyoruz.** Dönem projesi bu bölünmeyi gerektirecek kadar büyük değil ve erken bölmek gereksiz karmaşıklık yaratır. Ama gerçek bir kurumsal projede ilk göreceğiniz yapılardan biridir — bu yüzden burada.

---

8. Kaynakça

Resmî dokümantasyon *(hepsi `?view=aspnetcore-10.0` ile açılmalıdır)*

- MVC'ye genel bakış — https://learn.microsoft.com/en-us/aspnet/core/mvc/overview?view=aspnetcore-10.0
- Controller action'ları — https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/actions?view=aspnetcore-10.0
- Controller routing — https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/routing?view=aspnetcore-10.0

- Routing (çatı düzeyi) — https://learn.microsoft.com/en-us/aspnet/core/fundamentals/routing?view=aspnetcore-10.0
- Middleware — https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0
- Özel middleware yazmak — https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/write?view=aspnetcore-10.0
- Dependency Injection — https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection?view=aspnetcore-10.0
- Area'lar — https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/areas?view=aspnetcore-10.0

Desen ve kuram

- Trygve Reenskaug'un özgün MVC notu (1979) — Smalltalk-80 için yazılan ilk tanım
- Service Locator karşıt deseni — Martin Fowler, *Inversion of Control Containers and the Dependency Injection pattern* (2004)